

TypeScript Master Cheat Sheet

Type System, Interfaces, Generics, Utility Types & Advanced Guards

1. Primitive & Top/Bottom Types

- **Primitives:** `string`, `number`, `boolean`, `symbol`, `null`, `undefined`.
- **unknown (Top Type):** Type-safe alternative to `any`. Requires type narrowing before operating.
- **never (Bottom Type):** Represents values that never occur (e.g., throwing functions or exhaustive switch arms).

```
function processValue(val: unknown) {  
  if (typeof val === "string") console.log(val.toUpperCase()); // Narrowed to string  
}
```

2. Interfaces vs Types

```
// Interface: Open for declaration merging, ideal for objects & OOP contracts  
interface User {  
  readonly id: number;  
  name: string;  
  email?: string; // Optional property  
}  
  
// Type Alias: Flexible, supports Unions, Primitives & Tuples  
type Role = 'Admin' | 'User' | 'Guest';  
type UserWithRole = User & { role: Role }; // Intersection
```

3. Generics

```
// Generic Function with Constraint  
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] {  
  return obj[key];  
}  
  
// Generic Interface  
interface ApiResponse<TData> {  
  data: TData;  
  status: number;  
  error?: string;  
}
```

4. Built-In Utility Types

```
interface Task {  
  id: string;  
  title: string;  
  completed: boolean;  
}
```

```
type PartialTask = Partial<Task>; // All optional
type StrictTask = Readonly<Task>; // All immutable
type TaskPreview = Pick<Task, 'id' | 'title'>; // Select subset
type TaskMeta = Omit<Task, 'completed'>; // Exclude keys
type TaskRecord = Record<string, Task>; // Key-value map
```

5. Type Narrowing & Custom Guard Predicates

```
interface Circle { kind: 'circle'; radius: number; }
interface Square { kind: 'square'; side: number; }
type Shape = Circle | Square;

// Custom Type Guard (is operator)
function isCircle(shape: Shape): shape is Circle {
  return shape.kind === 'circle';
}

// Exhaustive Switch Pattern
function getArea(shape: Shape): number {
  switch (shape.kind) {
    case 'circle': return Math.PI * shape.radius ** 2;
    case 'square': return shape.side ** 2;
    default:
      const _exhaustiveCheck: never = shape;
      return _exhaustiveCheck;
  }
}
```